CS 61A — Week 3

Lambda expressions, currying, functional abstraction; Midterm 1 review (Lectures 6–8, Composing Programs 1.6)

§1 Facts used from Weeks 1–2

(W1) Call expression: evaluate operator, then operands left to right, then apply. `print` returns `None`.

(W2) `and`/`or`: the value is the last operand evaluated; `and` stops at the first false value, `or` at the first true value. False values: `False`, `0`, `None`, `''`.

(W3) A function's parent is the frame in which it was defined. A new frame's parent is the parent of the function called. Lookup: local frame, then parent, ..., then global.

(W4) A name in a function body is looked up when the body runs, not when the function is defined.

(W5) `summation(n, term)` adds `term(1)`, ..., `term(n)`. Digit loop: `while n > 0`, `n % 10`, `n // 10`.

§2 This week

§2.1 Lambda expressions (Lecture 6)

The question. To sum $\frac{1}{2} + \frac{1}{4} + \ldots$ with `summation` we need a function for the k-th term. Must we stop, write a `def`, and invent a name for a function used once?

```
summation(5, lambda x: pow(1/2, x))
```

- `lambda x:  pow(1/2, x)` is an *expression* whose value is a function with formal parameter `x` that returns the value of `pow(1/2, x)`.
- The body is a **single expression**. There is **no `return` keyword**; the value of the expression is returned.
- Being an expression, it can be an operand, a return value, or the right side of an assignment: `term = lambda x:  pow(1/2, x)`.
- Every lambda expression can be rewritten with `def`. Not every `def` can be written as a lambda (a lambda body cannot contain statements such as `while` or assignment).
- The body is not evaluated when the lambda expression is evaluated; only when the resulting function is called. (Same as `def`.)
- Several parameters: `lambda x, y:  x + y`. None: `lambda:  42`.

Two differences from `def`. (1) A `def` binds a name; a lambda expression binds nothing by itself. (2) A function made by `def` knows its name; a lambda function is shown as `func λ(x) <line N> [parent=...]` in diagrams, and its frames are labelled `λ <line N>`.

Reading nested lambdas. `lambda f:  lambda x:  f(f(x))` is a function of `f` whose body is another lambda expression: it *returns a function* of `x`. The lecture's example:

```
>>> (lambda f: lambda x: f(f(x)))(lambda y: y * y)(3)
81
```

1. It is `twice(square)(3)` written without names: the first lambda is `twice`, the second argument-lambda is `square`.
2. Calling the first lambda with `square` returns the function `lambda x: square(square(x))`.
3. Calling that with `3` gives `square(9)`, which is `81`.

§2.2 Environments with lambda (Lecture 6)

The question. A lambda function, like any function, may use names that are not its parameters. Which frame supplies them?

The same rule as before (W3): the parent of a lambda function is the frame in which the lambda expression was *evaluated*. The lecture's demonstration uses one name, `common_ratio`, bound in four places:

```
common_ratio = 1/2
def get_term(common_ratio):
    def term(k):
        return pow(common_ratio, k)
    return term
common_ratio = 1/3
term = get_term(common_ratio)

def summation(n, term):
    common_ratio = 1/4
    total, k = 0, 1
    while k <= n:
        total, k = total + term(k), k + 1
    return total
common_ratio = 1/5
print(summation(5, term))
```

1. `get_term(common_ratio)` is called when the global `common_ratio` is `1/3`. Its frame `f1` binds the *parameter* `common_ratio` to `1/3`. The inner `term` has parent `f1`.
2. Later the global value becomes `1/5`, and the frame of `summation` has its own `common_ratio = 1/4`.
3. When `term(k)` runs, its frame's parent is `f1`. Lookup of `common_ratio` goes: `term`'s frame (absent), then `f1`: `1/3`. The frame of `summation`, where the call happens, is not in this environment at all.
4. Result: $\frac{1}{3} + \frac{1}{9} + \frac{1}{27} + \frac{1}{81} + \frac{1}{243} \approx 0.4979$.

> **The idea**
>
> The caller's frame is never consulted. An environment is the chain *local frame* → *parent* → *parent's parent* → ⋯ → *global*, and parents are fixed where functions are defined. If you find yourself looking in the frame of the function that made the call, you have left the chain.

§2.3 Currying (Lecture 6)

Currying converts a function of several arguments into a chain of functions that each take one argument: `pow(1/2, 5)` becomes `curry(pow)(1/2)(5)`.

```
def curry(f):
    def g(x):
        def h(y):
            return f(x, y)
        return h
    return g
```

Equivalently `curry = lambda f: lambda x: lambda y: f(x, y)`.

> **The idea**
>
> Why would anyone want this? `summation` needs a *one*-argument function, and `pow` takes two. Supplying the first argument now and the second later bridges the gap: `curry(pow)(3)` is the one-argument function $k \mapsto 3^k$, so `summation(5, curry(pow)(3))` is $3 + 9 + 27 + 81 + 243 = 363$.
>
> How to read `curry(pow)(3)(k)`: three calls, left to right. `curry(pow)` returns `g` (remembering `f = pow`). `g(3)` returns `h` (remembering `x = 3`). `h(k)` returns `pow(3, k)`.

Lecture 8's exercise: `reverse`. `reverse = lambda f: lambda x, y: f(y, x)` swaps the argument order, so `reverse(pow)(2, 3)` is `pow(3, 2)`. Define `square` and `cube` in one line each without `lambda` or `**`:

1. We want a one-argument function $x \mapsto$ `pow(x, 2)`. The argument we want to fix is the *second* one of `pow`.
2. `curry` fixes the *first* argument. So first swap the order: `reverse(pow)` takes (exponent, base).
3. `square = curry(reverse(pow))(2)` and `cube = curry(reverse(pow))(3)`. Check: `square(3)` is `reverse(pow)(2, 3)`, which is `pow(3, 2) = 9`.

§2.4 Zero-argument functions (Lecture 6)

- A function may take no arguments: `def f(): ...` or `lambda: ...`; it is called with `f()`.
- Use: to *delay* a computation, or to represent something that gives a possibly different result at every call. In the Hog project a die is a zero-argument function; calling it rolls it.
- `f` and `f()` are different things: the first passes the function along, the second runs it now. Passing `dice` lets the receiver roll as many times as it wants; passing `dice()` passes one number.

§2.5 Conditional expressions and `bool` (Lecture 7)

- `<a> if <condition> else <b>`: evaluate the condition; if true the value is that of `<a>`, otherwise that of `<b>`. Only one of `<a>`, `<b>` is evaluated. It is an expression, so it can appear inside a lambda body.
- `and`/`or` always give the value of the left or the right expression: `(3 and 4) - 5` is `4 - 5 = -1` (Fall 2022 Midterm 1, Q1(a)).
- `bool(x)` is `True` for a true value and `False` for a false value: `bool(0)`, `bool(0.0)`, `bool('')`, `bool(False)` are `False`; `bool(-1)`, `bool(' ')` (a space is a nonempty string) are `True`; `bool(print('fool'))` displays `fool` and is `False`.

§2.6 Environment diagram review (Lecture 7)

There are exactly three ways a name gets bound: an assignment statement (current frame); a `def` statement (current frame); a call, which binds the parameters in the new local frame. (An `import` also binds names in the current frame.)

Worked exam question (Fall 2022 Midterm 1, Q2, from Lecture 7).

```
def f(x):
    y = max(x * x, 3 * x)
    def zero(t):
        if t(x) > 0:
            return y
        return 0
    return zero

y = 1
while y < 10:
    if f(y)(lambda z: z - y + 10):
        max = y
    y = y + 1
```

This ends with the error `'int' object is not callable`. Why?

1. First pass, global `y` is `1`. `f(1)`: frame `f1`, `x` is `1`; `y` in `f1` is `max(1, 3) = 3`; `zero` has parent `f1`; returns `zero`.
2. The lambda expression is evaluated in the global frame, so its parent is global.
3. `zero(λ)`: frame `f2` with parent `f1`. `t(x)` calls the lambda with `z = 1`; its body `z - y + 10` finds `y` in the *global* frame (`1`): value `10 > 0`. So `zero` returns the `y` of `f1`, which is `3`.
4. `3` is a true value, so `max = y` runs in the global frame: **the global name `max` is now the integer `1`.**
5. Second pass, `y` is 2: `f(2)` evaluates `max(x * x, 3 * x)`. `max` is not in the local frame; it is found in the global frame, where it is `1`. Calling `1` is the error.

The built-in `max` still exists, but the global binding is found first. Three different `y`'s and a rebinding of a built-in name: the diagram keeps them apart.

§2.7 Writing functions on an exam (Lectures 7–8)

The lectures' procedure for a fill-in-the-blank implementation question.

1. Read the description. Verify the doctest examples by hand and pick one simple example to work with.
2. Describe in plain English a process that computes the output from the input in simple steps.
3. Read the template. Annotate each name in it with its value for your example. Decide what any additional names mean.
4. Write the code. Ask: can I stop early? What kind of value must be returned?
5. Check: did you return the right thing? Run the other examples through your code.

Worked: `nearest_prime` (variant of Fall 2022 Midterm 1 Q3(b), Lecture 7). Return the prime nearest to `n` (`n > 5`); in a tie the larger one. `is_prime` is given.

```
def nearest_prime(n):
    k = 0
    while True:
        if ______:
            return ______
        if k > 0:
            ______
        else:
            ______
```

1. Example `n = 21`: candidates in order of distance, larger first on ties: 21, 22, 20, 23. `23` is prime.
2. So `k` is an offset taking the values `0, 1, -1, 2, -2, 3, ...`, and the candidate is `n + k`. First blanks: `if is_prime(n + k):  return n + k`.
3. How does `k` move? From a positive `k` go to `-k` (same distance, other side). From `0` or a negative `k` go to `-k + 1` (next distance, larger side first).

```
        if is_prime(n + k):
            return n + k
        if k > 0:
            k = -k
        else:
            k = -k + 1
```

Worked: `all_digits` and `prefix_digits` (Spring 2025 Midterm 1 Q3(a,b), Lecture 8). `all_digits(n, cond)` returns whether `cond(d)` is true for every digit `d` of `n`.

```
def all_digits(n, cond):
    while n:
        if not cond(n % 10):
            return False
        n = n // 10
    return True
```

- "Can you stop before going through all the digits?" Yes: one failing digit decides the answer, so `return False` inside the loop. `True` can only be returned after the loop.

A *prefix* of `n` is `n // pow(10, p)`: 3456 has prefixes 3456, 345, 34, 3, 0. `prefix_digits(n, cond)` returns the largest prefix all of whose digits satisfy `cond`. Given the helper

```
def process(n, check):
    while n:
        if check(n):
            return n
        n = n // 10
    return 0
```

1. What does `process` do? It tries `n`, then `n // 10`, ... (the prefixes, largest first) and returns the first one for which `check` is true.
2. So `check` must take a whole prefix `k` and say whether all of its digits satisfy `cond`. That is `all_digits(k, cond)`.
3. `return process(n, lambda k:  all_digits(k, cond))`. The lambda adapts a two-argument function to the one-argument shape `process` expects, and it finds `cond` in its parent frame.

Shortcuts

- **Lambda**: no name, no `return`, one expression. Rewrite it as a `def` in the margin if the nesting is confusing.
- **Count the arrows of calls**: in `f(a)(b)(c)` each pair of parentheses is one call, applied left to right to the result of the previous one.
- **Adapting a function to a required shape** is the commonest blank: `lambda k: g(k, extra)` or `lambda k: term(2 * k - 1)`.
- **Which frame?** For any name in a body: local frame, then the frame where the function was defined, and so on. Never the caller's frame.
- **Early return**: "for every" questions return `False` inside the loop and `True` after it; "find the first" questions return the value inside the loop.
- `while True` with a `return` inside: use it when the stopping point is discovered in the middle of the body.
- **Rebinding a built-in** (`max = y`, `print = ...`) is a classic exam device: track it like any other name.
- **Running maximum without** `if`: `largest = max(largest, value)`.

Traps

- `lambda x: return x` is a syntax error.
- `f = lambda: 5` then `f` is a function; `f()` is `5`.
- A lambda's body sees the *current* value of outside names when it is called (W4), not their value when the lambda was created.
- `curry(pow)(2, 3)` is an error: after currying, arguments are supplied one call at a time.
- `bool(' ')` is `True`; `bool('')` is `False`; `bool(0.0)` is `False`.
- In `a if c else b`, the condition is in the middle.

§3 Problem set

P1–P5 are original. E1–E2 are from the Fall 2026 Midterm 1 (your exam), with the published answers in the solutions. For a timed practice exam before a midterm, see `exercises/README.md`.

P1 (lambda values). What is displayed?

```
>>> f = lambda x, y: x - y
>>> g = lambda h: lambda x: h(x, x + 1)
>>> g(f)(10)
>>> (lambda: 3)() + (lambda x: x * 2)(4)
>>> k = lambda x: print(x)
>>> k(5) or 'done'
```

P2 (rewrite). (a) Rewrite with `def` statements and no lambda: `make = lambda a: lambda b: lambda c: a * b + c`. (b) Rewrite as a single lambda expression:

```
def pick(test):
    def choose(x, y):
        if test(x):
            return x
        return y
    return choose
```

P3 (which frame). What is displayed?

```
n = 1
def outer(n):
    return lambda k: k + n
def caller(f):
    n = 100
    return f(10)
g = outer(5)
n = 50
print(caller(g), caller(lambda k: k + n))
```

P4 (currying). With `curry` and `reverse` as in the text and `from operator import sub`: give the values of `curry(sub)(10)(3)`, `curry(reverse(sub))(10)(3)`, and `summation(3, curry(reverse(sub))(1))`.

P5 (write by hand, exam style). Implement `count_where(n, cond)`, the number of digits `d` of the positive integer `n` with `cond(d)` true. Then, using `count_where` and one lambda and no loop, implement `has_digit(n, d)`, which returns whether digit `d` occurs in `n`.

E1 (Fall 2026 Midterm 1, Question 1, 6 points). What would Python display on the lines marked (a)–(d)?

```
>>> z = 'but'
>>> f = lambda g: g(z)
>>> z = 'so'
>>> x, y = print(1 or 2), (3 and 0)
(a) _____
>>> print(print(x, '=', y), '+', f(print), '= zero?')
(b) _____
(c) _____
(d) _____
```

E2 (Fall 2026 Midterm 1, Question 4.1–4.2: Space Between). You may not write `if`, `else`, `[`, `]`, `for` or `range`. `gap(n, term)` returns the largest absolute difference between consecutive terms of `term(1)`, ..., `term(n)` (`n > 1`): `gap(5, lambda x: x * x)` is 9; `gap(6, lambda x: x % 4)` is 3. `odd_gap(n, term)` does the same for `term(1)`, `term(3)`, `term(5)`, ... and must call `gap`: `odd_gap(5, lambda x: x * x)` is 16.

```
def gap(n, term):
    k, largest = 1, 0
    while ______:           # (a): k < n - 1,  k < n,  k < n + 1,  or  k < n + 2
        largest = ______    # (b)
        k = k + 1
    return largest

def odd_gap(n, term):
    return gap((n + 1) // 2, ______)    # (c)
```